

# Module 04: Tool Calling & Function Callbacks

## Connecting LLMs to Enterprise Databases, APIs, and Spring Services

**Instructor:** AI Curriculum Engineering Team

**Tech Stack:** Java 25/27 (LTS), Spring Boot 3.4/4.x, Spring AI 1.0+, Ollama

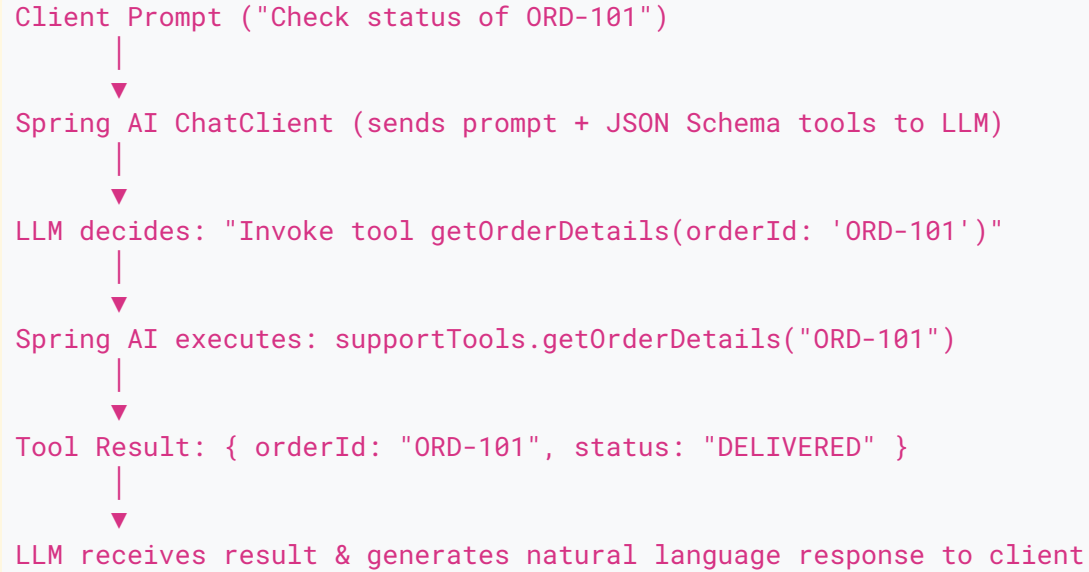
# 1. Why Tool Calling?

Foundation models are cut off from external state:

- They cannot query your live Postgres database.
- They cannot check real-time stock levels or execute refunds.
- They hallucinate when asked for private corporate facts.

**Tool Calling** enables models to output a structured intention to invoke code, pause generation, wait for the application to return the execution result, and synthesize the final answer.

## 2. Tool Execution Lifecycle



## 3. Declaring Tools with `@Tool`

In modern Spring AI 1.x, declare tools directly on any Spring Bean:

```
@Service
public class OrderTools {

    @Tool(description = "Lookup order by its ID string, e.g. ORD-101")
    public OrderRecord getOrderDetails(String orderId) {
        return orderRepository.findById(orderId);
    }
}
```

Spring AI automatically translates the method parameters and `@JsonPropertyDescription` annotations into OpenAPI-compliant JSON Schemas.

## 4. Fluent ChatClient Tool Binding

Bind tools fluently during prompt invocation:

```
@Service
public class SupportAgentService {

    private final ChatClient chatClient;
    private final OrderTools orderTools;

    public String handleCustomer(String message) {
        return chatClient.prompt()
            .tools(orderTools) // LLM can now call orderTools dynamically!
            .user(message)
            .call()
            .content();
    }
}
```

## 5. Homework 04 Assignment & Hands-on Lab

### "Autonomous Customer Support Ticket Triage Agent"

1. Open `module-04-tool-calling-callbacks/homework/starter`.
2. Inspect `OrderRefundService.java`.
3. Annotate `checkEligibility` with `@Tool`.
4. Bind the tool to `ChatClient` in `processRefundRequest`.
5. Run `mvn test` to verify complete execution.